

Family Tree Management System

A Console-Based C++ Application for Modelling Multi-Generational Family Relationships

C++17

Linked Data Structures

Recursion

Menu-Driven CLI

XML Export

PREPARED BY
Taqi

INSTITUTION
COMSATS University Islamabad, Lahore Campus

PROGRAMME
BS Software Engineering

Table of Contents

Chapter 1	Introduction	1
Chapter 2	Project Objectives	2
Chapter 3	Project Scope	3
Chapter 4	System Design	4
	4.1 Overall System Architecture	
	4.2 Family Forest Concept	
	4.3 Program Workflow	
	4.4 Relationship Representation	
	4.5 Navigation Flow	
	4.6 Menu Hierarchy	
	4.7 System Flow Diagram	
Chapter 5	Data Structures	7
	5.1 Person Structure	
	5.2 Child Representation (Intrusive Sibling List)	
	5.3 FamilyTree Structure	
	5.4 HistoryNode Structure	
	5.5 SearchNode Structure	
	5.6 Global Variables	
Chapter 6	Working of the Complete System	11
	6.1 – 6.19 Every functional module (menu → export)	
Chapter 7	Relationship Constraints	21
Chapter 8	Input Validation	24
Chapter 9	Design Decisions	26
Chapter 10	Complexity Analysis	30
Chapter 11	Libraries Used	32
Chapter 12	Conclusion	33

Introduction

The **Family Tree Management System** is a console-based application written in C++ that allows a user to build, connect, and explore multi-generational family relationships. Rather than storing a family as a flat list of records, the system models it as a collection of linked `Person` nodes, each carrying pointers to its father, mother, spouse, and children. This turns every family into a navigable graph rather than a static table, which is what allows features such as "View Parents", "View Siblings", "Previous Person" navigation, and name/ID search to work purely through pointer traversal instead of repeated database-style lookups.

The project was built as an exercise in applying core data-structure concepts — linked lists, recursive tree traversal, and stack-based navigation history — to a real, interactive problem: representing the irregular, cyclic-looking, many-to-many nature of human family relationships using strictly acyclic, pointer-based structures. It is entirely self-contained: no external database or file storage is required during runtime, and the only persistent artifact the system produces is an on-demand XML export of a family tree.

Project Objectives

- Represent an arbitrary number of independent family lines ("family forest") simultaneously, while still allowing relationships to be created *between* people from different lines (e.g. marrying two people who each started as separate tree roots).
- Provide full CRUD-style relationship management: registering fathers, mothers, spouses, and children, each governed by explicit validation rules (one father, one mother, one active spouse, no duplicate children, no self-relationships).
- Support both *creation* of a new relative and *linking* of an already-registered person, so the same individual can correctly appear in more than one family role (e.g. a person who is simultaneously someone's child and someone else's spouse).
- Offer intuitive, menu-driven navigation between people (parents, children, siblings, spouse) with the ability to step back through recently visited people via a "Previous Person" history stack.
- Provide search by unique ID and by name (case-insensitive), returning every matching person in the system regardless of whether that person owns their own family tree.
- Allow lifecycle updates — renaming a person, marking a person deceased, and ending a marriage (divorce) — with rules that keep the rest of the graph consistent.
- Export any family tree to a well-formed, human-readable XML document for external viewing or archiving.

Project Scope

In scope:

- Single-session, in-memory management of any number of family trees, connected people, and relationships.
- Parent (father/mother), spousal, and child relationships, including divorce and deceased-status handling.
- Derived views that are not separately stored: siblings (derived from a shared parent's child list) and full family-tree printing (derived from recursive traversal).
- ID-based and name-based search across the entire connected family graph, not just a single tree.
- Navigation history for moving backward through recently visited people.
- XML export of a chosen family tree.

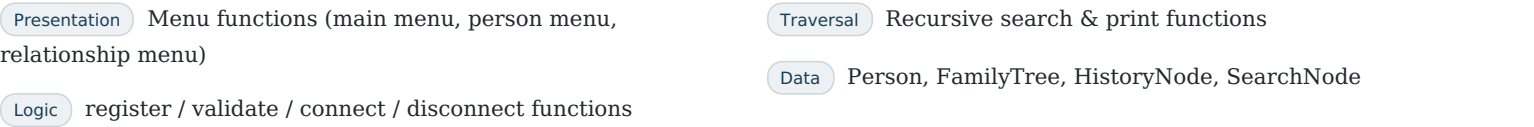
Out of scope:

- Persistent storage between program runs — all data is held in memory and is lost on exit (XML export is the only durable artifact).
- Concurrent/multi-user access — the system is a single-threaded, single-session console tool.
- Graphical rendering of the tree — the tree is printed as indented text, not drawn visually.
- Complex genealogical edge cases such as adoption chains, polygamous marriages, or same-sex parent pairs, which are outside the relationship rules currently enforced.

System Design

4.1 Overall System Architecture

The system follows a straightforward layered console-application architecture built entirely in a single translation unit. There is no separate persistence layer; the "database" is simply the graph of dynamically allocated `Person` objects reachable from a set of tree roots.



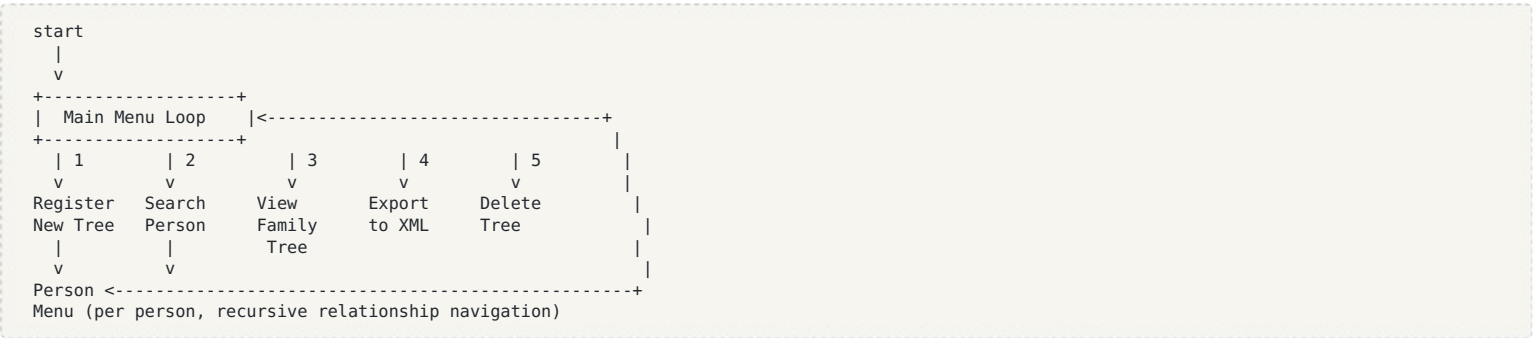
4.2 Family Forest Concept

Because a program may need to track several unrelated families at once, the system does not use a single tree — it uses a **forest**: a singly linked list of `FamilyTree` nodes, each holding a pointer to that family's root `Person`. New trees are appended to `familyForestHead` whenever the user registers a brand-new root person, or when a newly created relative is explicitly given "their own tree" instead of only being linked into an existing one.

IMPORTANT NUANCE

Being a "root" is a bookkeeping label, not a structural boundary. A father, mother, or spouse created without their own tree is still fully connected to the graph through pointers — they are simply not the entry point of any `FamilyTree` node. Earlier revisions of the search functions only walked downward from tree roots and therefore could not find such people; the current implementation performs a full graph walk (father, mother, spouse, and children) from every root so that *every* connected person is reachable, regardless of whether they own a tree.

4.3 Program Workflow



4.4 Relationship Representation

Every relationship a person can have maps to exactly one pointer (or one small linked chain) inside that person's own structure — there is no separate, external "relationships table":

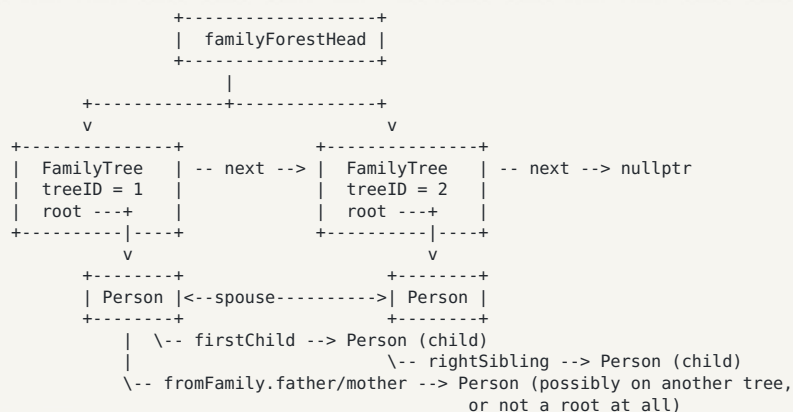
Relationship	Stored As	Cardinality
Father	<code>fromFamily.father</code>	0 or 1
Mother	<code>fromFamily.mother</code>	0 or 1
Spouse	<code>extendFamily.spouse</code>	0 or 1 (monogamous, mutually assigned)
Children	<code>extendFamily.firstChild</code> / <code>lastChild</code> chain	0 or many
Siblings	<i>not stored</i> — derived from a shared parent's child chain	0 or many

4.5 Navigation Flow

The person-menu system is recursive: `personMenu(Person*& currentPerson)` takes its subject *by reference*, so relationship views (View Parents / Children / Siblings / Spouse) can redirect the same menu session to a different person simply by reassigning the reference, without opening a brand-new nested menu call. A global `HistoryNode` stack records where the user *came from* whenever a genuinely new person menu is opened (for example, right after creating a child), so option 7, "Previous Person", can unwind that path one step at a time.

4.6 Menu Hierarchy

4.7 System Flow Diagram



Data Structures

5.1 Person Structure

Purpose: The single unit of the entire system. Every registered individual — root, child, spouse, or linked relative — is one `Person` node.

```
struct Person
{
    int id;
    string name;
    string gender;

    FromFamily fromFamily;    // father, mother
    ExtendFamily extendFamily; // spouse, firstChild, lastChild

    Person* leftSibling;
    Person* rightSibling;

    LifeStatus status;        // ALIVE / DECEASED
    bool visited;             // transient traversal-guard flag
};
```

Complete Member Explanation

Member	Role
id	Unique identifier assigned sequentially from a global counter; used by ID search and equality checks.
name, gender, status	Core attributes used throughout validation (e.g. father must be male, spouse must be alive).
fromFamily	The two parent pointers — see 4.4.
extendFamily	Spouse pointer and the head/tail of this person's child chain.
leftSibling / rightSibling	Position of <i>this</i> person inside its parent's child chain (see 5.2).
visited	A transient boolean used only during graph search to prevent infinite recursion when the search follows parent, spouse, and child pointers in all directions; reset to <code>false</code> immediately after each search completes.

Parent Pointers

`fromFamily.father` and `fromFamily.mother` are plain single pointers rather than a list. This is a deliberate structural enforcement of the "one father / one mother" rule — the data structure itself makes a second father or mother physically inexpressible, in addition to the explicit `hasFather()` / `hasMother()` validation checks performed before assignment.

Spouse Pointer

`extendFamily.spouse` is likewise a single pointer, assigned symmetrically (`husband->spouse = wife; wife->spouse = husband;`) by `connectSpouses()`. This structurally enforces monogamy: a person cannot be pointed to as a spouse by two different people at once without first being disconnected.

Child Linked List

Children are *not* stored in a `vector` or array. Each parent keeps only `firstChild` and `lastChild`; children themselves are threaded together via their own `leftSibling` / `rightSibling` pointers, forming a doubly linked list rooted at the parent. See 5.2 for full detail.

ExtendFamily Structure

```
struct ExtendFamily
{
    Person* spouse;
    Person* firstChild;
    Person* lastChild;
};
```

FromFamily Structure

```
struct FromFamily
{
    Person* father;
    Person* mother;
};
```

Memory Representation



Complexity Benefits

Keeping both a `firstChild` and a `lastChild` pointer (rather than only `firstChild`) turns child insertion from an **O(n)** walk-to-the-end operation into an **O(1)** append, at the cost of two extra pointer writes per insertion.

DESIGN JUSTIFICATION

An intrusive sibling list embedded directly in `Person` avoids a second class of wrapper nodes purely for child bookkeeping, keeps every relationship reachable through the same `Person*` type, and lets the same recursive traversal pattern (`firstChild` → walk via `rightSibling`) be reused for printing, XML export, and search.

5.2 Child Representation (Intrusive Sibling List)

The system does not use a dedicated wrapper structure (such as a generic `ChildNode`) to hold child references. Instead, the sibling-list pointers live directly inside `Person` itself — this is known as an **intrusive linked list**.

Purpose

Represent, for a single parent, the ordered set of that parent's children without a separate collection object.

Node Members

`Person::leftSibling`, `Person::rightSibling` — every person doubles as its own sibling-list node.

Child Storage

The parent stores only two pointers, `firstChild` and `lastChild`; the children themselves carry the ordering.

Traversal

```
Person* child = parent->extendFamily.firstChild;
while (child != nullptr) {
    // process child
    child = child->rightSibling;
}
```

Insertion

New children are always appended after `lastChild`, linking `lastChild->rightSibling` forward and the new child's `leftSibling` backward, then updating `lastChild`.

Complexity

Operation	Complexity
Append child	O(1)
Traverse all children	O(k), k = number of children
Duplicate-child check	O(k)

DESIGN JUSTIFICATION

A separate `ChildNode` wrapper would need its own allocation per child *and* its own pointer back to the actual `Person`, doubling allocations and adding an extra indirection on every traversal for no functional benefit, since a child can never simultaneously belong to two different sibling lists.

5.3 FamilyTree Structure

```
struct FamilyTree
{
    int treeID;
    Person* root;
    FamilyTree* next;
};
```

Purpose: Represents one independently rooted family line and links it into the forest.

Root Pointer: Entry point for all root-down operations — printing, XML export, and the starting point of every graph search.

Next Pointer: Chains every tree into a singly linked list headed by `familyForestHead`, so the system can support an unbounded number of unrelated families in the same run.

Operation	Complexity
Register new tree	O(1) (inserted / appended at the forest list)
Locate a tree by ID	O(t), t = number of trees
Delete a tree	O(n) to free all nodes + O(t) to unlink

DESIGN JUSTIFICATION

Modelling "the whole system" as a forest rather than a single mega-tree keeps unrelated ancestries cleanly separated for printing/export purposes, while cross-tree relationships (e.g. marrying into another family) remain possible because relationships are ordinary pointers that are not constrained to stay within one tree.

5.4 HistoryNode Structure

```
struct HistoryNode
{
    Person* currentPerson;
    HistoryNode* next;
};
```

Purpose: Backing store for the "Previous Person" feature — a LIFO stack of people the user has navigated away from.

Navigation Stack: Implemented as a singly linked list with a single global head pointer, `historyTop`.

Push Operation: Called with the person being navigated *away from*, immediately before the menu is redirected to a new person — for example, right before opening a newly created child's menu, the parent is pushed.

Pop Operation: Removes and returns the top entry; if the stack is empty, returns `nullptr`, and "Previous Person" reports that none is available instead of crashing or silently misbehaving.

Operation	Complexity
Push	O(1)
Pop	O(1)
Clear (on fresh entry point)	O(d), d = current stack depth

DESIGN JUSTIFICATION

A stack is the natural fit for "undo the last navigation step" semantics. To prevent stale entries from an earlier, unrelated browsing session from resurfacing, the stack is explicitly cleared whenever a menu is opened from a context with no logical predecessor (a brand-new tree root, or a person reached via search) rather than only ever being pushed to.

5.5 SearchNode Structure

```
struct SearchNode
{
    Person* person;
    SearchNode* next;
};
```

Purpose: A disposable, single-use linked list used to accumulate the results of one search operation (by ID or by name) before presenting them to the user.

Temporary Search Results: Built up by `addSearchResult()` during a recursive graph walk, which also guards against adding the same person twice (relevant because a single person can now be reached through more than one path — e.g. as both a tree root and someone's spouse).

Linked List Representation: Head/tail pointers allow O(1) append, mirroring the `Person` child-chain design.

Memory Cleanup: `destroySearchResults()` walks and frees every node once the user has made a selection (or cancelled), so no search-list memory persists between searches.

Operation	Complexity
Add result (with duplicate check)	O(r), r = results so far
Destroy list	O(r)

DESIGN JUSTIFICATION

Search results are collected in a completely separate structure from the family graph itself, so browsing or discarding search results never risks mutating real relationship pointers.

5.6 Global Variables

Variable	Type	Purpose
<code>familyForestHead</code>	<code>FamilyTree*</code>	Entry point to every registered family line in the system.
<code>historyTop</code>	<code>HistoryNode*</code>	Top of the navigation-history stack used by "Previous Person".
<code>nextPersonID</code>	<code>int</code>	Monotonically increasing counter handed out to every new <code>Person</code> , guaranteeing unique IDs without a lookup.
<code>nextTreeID</code>	<code>int</code>	Same role as <code>nextPersonID</code> , but for <code>FamilyTree</code> identifiers.

NOTE ON "CURRENTPERSON"

The active person being viewed is *not* kept as a global variable. It is passed as a reference parameter, `Person*& currentPerson`, through the recursive `personMenu()` call chain instead. This means each nested menu session has its own independent notion of "current person" rather than sharing one piece of global mutable state, which avoids an entire category of bugs where one navigation path could unexpectedly overwrite another's context.

DESIGN JUSTIFICATION

Only state that is genuinely program-wide (the forest, the navigation stack, and the ID counters) is global; everything else is passed explicitly, keeping the call graph easy to reason about.

Working of the Complete System

Each module below is described using the same lens: what it is for, which structures it touches, how it works step by step, what pointers change, how it validates its input, notable edge cases, its complexity, and why it was built that way.

6.1 Main Menu

PURPOSE

Program's single entry loop; routes to every top-level feature.

DATA STRUCTURES USED

`familyForestHead` (indirectly, via the features it calls).

PROCEDURE

Displays five options plus Exit; reads and validates a bounded integer choice; dispatches via `switch`; loops until `0`.

INTERNAL WORKING

Delegates entirely to `registerNewFamilyTree()`, `searchMenu()`, `viewFamilyTree()`, `createXML()`, and `deleteTreeMenu()`; the loop itself owns no domain logic.

POINTER / RELATIONSHIP UPDATES

None directly.

VALIDATION

`inputChoice(0, 5, ...)` rejects out-of-range or non-numeric input and reprompts.

SPECIAL CASES

On exit (`0`), the history stack is cleared and every tree is freed before the process ends.

COMPLEXITY

O(1) per iteration (excluding the called feature).

DESIGN JUSTIFICATION

A single dispatch loop keeps the top-level control flow in one place, so every feature is reachable from one predictable entry point.

6.2 Create Family Tree

PURPOSE

Allocate a brand-new `FamilyTree` and attach a root person to it.

DATA STRUCTURES USED

`FamilyTree`, `Person`.

PROCEDURE

`createTreeForPerson(root)` validates the person via `validateTreeCreation()`, allocates a `FamilyTree`, assigns it the next tree ID, sets `root`, and links it into `familyForestHead`.

INTERNAL WORKING

Used both when registering a fresh root person from the main menu, and when the user answers "Y" to "Create a new Family Tree for this Person?" while registering a father, mother, or spouse.

POINTER / RELATIONSHIP UPDATES

`newTree->next = familyForestHead; familyForestHead = newTree;`

VALIDATION

`validateTreeCreation()` rejects a null person.

SPECIAL CASES

A relative can be created *without* a tree of its own — it still exists and is still reachable through the graph, just not as a search/print/export entry point.

COMPLEXITY

O(1).

DESIGN JUSTIFICATION

Making tree-creation optional for every new relative lets the user decide which ancestries deserve their own top-level entry versus which are just "hanging off" an existing tree.

6.3 Register Root Person

PURPOSE

Collect name and gender for a brand-new root individual and start a tree from them.

DATA STRUCTURES USED

`Person`, `FamilyTree`.

PROCEDURE

Prompts for a validated name and gender, calls `createPerson()` to allocate the `Person` with a fresh ID, then `createTreeForPerson()`.

INTERNAL WORKING

On success, immediately offers to open the new person's menu via `askOpenPersonMenu()`.

POINTER / RELATIONSHIP UPDATES

None besides the new tree's root pointer.

VALIDATION

Name: alphabetic characters, spaces, apostrophes, hyphens only, non-empty. Gender: must be exactly "male" or "female".

SPECIAL CASES

A root person starts with no father, mother, spouse, or children — all four pointers default to `nullptr`.

COMPLEXITY

O(1).

DESIGN JUSTIFICATION

Reusing the same `createPerson()` factory for a root as for every other relative keeps ID assignment and struct initialization in exactly one place.

6.4 Register Father

PURPOSE

Give a person a father, either newly created or linked from an existing person.

DATA STRUCTURES USED

`Person.fromFamily.father`, child's sibling chain on the father's side.

PROCEDURE

Sub-menu offers "Create New Father" / "Link Existing Father" / "Back". Both paths converge on `establishParentRelationship(child, father)`.

INTERNAL WORKING

Creation path: collect a name, build a male `Person`, optionally give it its own tree. Linking path: `findExistingPerson()` (ID or name search) locates the target.

POINTER / RELATIONSHIP UPDATES

`child->fromFamily.father = father;` and the child is appended to the father's child chain via `addChildToParent()`.

VALIDATION

`validateFatherRegistration()`: rejects null persons, self-assignment, a deceased father, a child who already has a father, and a father whose gender is not male.

SPECIAL CASES

If the child already has a mother with an active spouse, the father registration deliberately does *not* auto-marry them — father/mother are independent relationship slots.

COMPLEXITY

Create path O(1); Link path bounded by the graph-search complexity in 6.9.

DESIGN JUSTIFICATION

Splitting "create" and "link" keeps the common case (a brand-new ancestor) fast and simple, while still supporting the more complex case of connecting two already-registered branches of the forest.

6.5 Register Mother

PURPOSE

Symmetric counterpart to 6.4 for the mother slot.

DATA STRUCTURES USED

`Person.fromFamily.mother`.

PROCEDURE / INTERNAL WORKING

Identical structure to Register Father: Create New Mother / Link Existing Mother / Back, converging on `establishParentRelationship()`.

POINTER / RELATIONSHIP UPDATES

`child->fromFamily.mother = mother;` plus child-chain insertion under the mother.

VALIDATION

`validateMotherRegistration()` mirrors the father checks, requiring female gender.

SPECIAL CASES

A mother linked from another tree keeps her original tree membership; only her pointers change, not her tree ownership.

COMPLEXITY

Same as 6.4.

DESIGN JUSTIFICATION

Keeping father and mother as two independent code paths (rather than one generic "add parent" with a type flag) keeps each validation function short and its gender rule explicit and easy to audit.

6.6 Register Child

PURPOSE

Add a new child under the current person (and their spouse, if married).

DATA STRUCTURES USED

`extendFamily.firstChild / lastChild`, sibling chain.

PROCEDURE

Collects name and gender, creates the child, calls `validateChildRegistration()`, appends the child under the current person via `addChildToParent()`, and — if the current person has a spouse — also appends the same child under the spouse.

INTERNAL WORKING

The child's own `fromFamily.father / mother` pointers are set based on the registering person's and their spouse's gender.

POINTER / RELATIONSHIP UPDATES

New child linked into up to two parents' sibling chains simultaneously, keeping both sides consistent.

VALIDATION

`validateChildRegistration()` rejects a null parent/child, self-assignment, and a duplicate child (checked via `childAlreadyExists()` walking the sibling chain).

SPECIAL CASES

If the person has no spouse, the child is registered under only that one parent — the other parent slot remains empty until later linked.

COMPLEXITY

O(k) for the duplicate check (k = existing children), O(1) for the append itself.

DESIGN JUSTIFICATION

Automatically mirroring the child under both parents when a spouse exists keeps the "one father, one mother" invariant true from the moment of creation, instead of requiring a second manual linking step.

6.7 Register Spouse

PURPOSE

Marry the current person to a new or existing person, with full handling of an already-existing marriage.

DATA STRUCTURES USED

`extendFamily.spouse` (bidirectional).

NEW PERSON

Collects a name, infers the required opposite gender, creates the `Person`, optionally gives them their own tree, and calls `connectSpouses()`.

EXISTING PERSON

`findExistingPerson()` locates the target across the entire connected graph (see Chapter 4.2), then the same `connectSpouses()` call is used.

ALREADY MARRIED PERSON

If either party already has a spouse, `resolveExistingMarriage()` is triggered, presenting the user with explicit choices rather than silently failing or silently overwriting.

DIVORCE

Option 2 of the resolution menu calls `disconnectSpouses()`, clearing both spouse pointers.

DECEASED SPOUSE

Option 1 marks the existing spouse deceased *and* disconnects the pointer, modelling widowhood before allowing the new marriage.

CANCEL OPERATION

Option 3 aborts without touching any pointer.

POINTER UPDATES

`husband->spouse = wife; wife->spouse = husband;`, set together so the relationship can never be one-sided.

RELATIONSHIP VALIDATION

`validateSpouseRegistration()`: rejects null/self, a deceased participant, an existing spouse on either side, and a same-gender pairing.

COMPLEXITY

Create path $O(1)$; link path bounded by graph search; divorce/deceased handling $O(1)$.

DESIGN JUSTIFICATION

Treating "already married" as a first-class, user-facing decision point (rather than an error message) reflects that divorce and widowhood are normal, expected life events the system needs to model, not exceptional failures.

6.8 Search by ID

PURPOSE

Locate one specific person anywhere in the connected family graph by their unique numeric ID.

DATA STRUCTURES USED

`SearchNode` (for the transient visited list), `Person.visited`.

PROCEDURE

`findPersonByID(id)` iterates every tree in the forest, calling `findPersonByIDRecursive()` on each root.

INTERNAL WORKING

The recursive search follows `father`, `mother`, `spouse`, and every child from the current node, marking each visited node so it is not revisited (a person's father and a person's spouse's father, for instance, may be the very same node reached two different ways).

POINTER / RELATIONSHIP UPDATES

None — read-only. `visited` flags are set during the walk and reset to `false` for every visited node once the search concludes, win or lose.

VALIDATION

Rejects `id <= 0` immediately.

SPECIAL CASES

Stops as soon as a match is found (short-circuit return), but still performs the visited-flag cleanup for whatever partial traversal occurred.

COMPLEXITY

$O(n)$ worst case, n = total connected people, since every person is visited at most once.

DESIGN JUSTIFICATION

A full graph walk (rather than a tree-only, root-down walk) is required precisely because a person can be connected to the system purely through a parent or spouse link, without being any tree's root.

6.9 Search by Name

PURPOSE

Locate every person whose name matches a query, case-insensitively, and let the user choose among the matches.

DATA STRUCTURES USED

`SearchNode` results list, `SearchNode` visited list, `Person.visited` .

PROCEDURE

`findPersonsByName(name)` lower-cases the query once, then walks every tree with the same father/mother/spouse/children graph traversal used for ID search, comparing each visited person's lower-cased name.

INTERNAL WORKING

Matches are collected into a `SearchNode` results list via `addSearchResult()` , which itself guards against adding the same person twice (possible now that a person is reachable via multiple relationship paths). Results are printed, and `chooseSearchResult()` lets the user pick one or cancel.

POINTER / RELATIONSHIP UPDATES

None — read-only.

VALIDATION

Name comparison is fully case-insensitive (`toLowerCase()` applied to both the query and every candidate name), so "sara", "Sara", "SARA", and "sArA" all match the same stored person.

SPECIAL CASES

If no results are found, the caller is informed directly rather than presenting an empty menu.

COMPLEXITY

O(n) to walk the graph + O(r²) worst case for the duplicate-guarded inserts (r = number of matches, typically small).

DESIGN JUSTIFICATION

Case-insensitive comparison matches how people naturally type names; combining it with the full graph walk (rather than tree-only) ensures the two most commonly reported "person not found" scenarios — wrong letter case, and the person only being linked, not rooted — are both eliminated at the source.

6.10 Person Menu

PURPOSE

The central hub for everything that can be done to or from one specific person.

DATA STRUCTURES USED

`Person*& currentPerson` (reference parameter), `HistoryNode` stack.

PROCEDURE

Loops, printing the person's details and eight options, dispatching to view/register/update/navigate functions.

INTERNAL WORKING

Because `currentPerson` is a reference, options 1–4 and 7 can redirect the *same* loop iteration to a different person by reassignment, rather than recursively opening a new menu call for every hop.

POINTER / RELATIONSHIP UPDATES

None directly — delegates to the relevant sub-feature.

VALIDATION

Guards against a `nullptr` subject on entry.

SPECIAL CASES

Option 8 ("Back") exits the current call frame, unwinding to whichever caller opened this particular menu instance (which may itself be a nested `personMenu` call from an earlier "Open Person's Menu" prompt).

COMPLEXITY

O(1) per menu iteration.

DESIGN JUSTIFICATION

Using a reference parameter instead of always recursing keeps ordinary relationship browsing flat and cheap, while genuinely new entry points (new tree, search result, freshly created relative) still get a real, separately poppable history entry.

6.11 View Person

PURPOSE

Render one person's core attributes in a consistent format, reused by nearly every other feature.

DATA STRUCTURES USED

`Person` (read-only).

PROCEDURE

`printPersonDetailed()` prints ID, name, gender, and status inside a bordered block.

INTERNAL WORKING

Called at the top of the Person Menu, after search, and after creating any new relative.

VALIDATION

Guards against a null person.

COMPLEXITY

O(1).

DESIGN JUSTIFICATION

Centralizing the display format avoids formatting drift between the dozen or so call sites that need to show a person's summary.

6.12 View Parents

PURPOSE

List the current person's father and/or mother and let the user jump to one.

PROCEDURE

Builds a small numbered list from whichever of `fromFamily.father / mother` are non-null, then returns the chosen `Person*` to the caller.

INTERNAL WORKING

Before returning, pushes the person being navigated *away from* onto the history stack, so "Previous Person" can return here.

SPECIAL CASES

If neither parent is registered, informs the user instead of showing an empty list.

COMPLEXITY

O(1).

DESIGN JUSTIFICATION

Returning the selected parent (rather than only printing it) is what allows Person Menu to redirect its own reference and support onward navigation.

6.13 View Children

PURPOSE

List the current person's children and let the user jump to one.

PROCEDURE

Walks `firstChild` → `rightSibling`, numbering each entry, then resolves the chosen index back to a `Person*`.

SPECIAL CASES

Empty child chain is reported explicitly ("No children registered").

COMPLEXITY

O(k), k = number of children.

DESIGN JUSTIFICATION

Same push-then-redirect pattern as View Parents keeps navigation history consistent across every relationship type.

6.14 View Siblings

PURPOSE

List everyone who shares at least one parent with the current person.

PROCEDURE

Walks the child chain of whichever parent(s) the current person has, skipping the current person itself, and offers the resulting list.

WHY SIBLINGS ARE DERIVED INSTEAD OF STORED SEPARATELY

A dedicated "sibling" pointer or list on `Person` would be redundant data that could drift out of sync with the parent/child pointers whenever a father or mother is later added, removed, or re-linked. Because every sibling relationship is fully implied by "shares a parent's child chain", deriving it on demand from data that already exists guarantees it can never become inconsistent — there is nothing to update when relationships change, because nothing extra was stored in the first place.

COMPLEXITY

$O(k)$, k = children of the shared parent(s).

DESIGN JUSTIFICATION

Trading a small amount of repeated computation for the complete elimination of a redundant, sync-prone field.

6.15 View Spouse

PURPOSE

Jump directly to the current person's spouse, if any.

PROCEDURE

Reads `extendFamily.spouse` directly — no search needed, since the pointer is already a direct reference.

SPECIAL CASES

Reports clearly if the person has no spouse rather than showing an empty selection menu.

COMPLEXITY

$O(1)$.

DESIGN JUSTIFICATION

The single-pointer spouse design (5.1) makes this the cheapest of all the relationship views.

6.16 Update Person

PURPOSE

Modify a person's mutable attributes after creation: name, life status, and marital status.

PROCEDURE

Sub-menu with three actions plus Back, each delegating to a small dedicated function (`updatePersonName()`, `updateLifeStatus()` / `markPersonDeceased()`, `divorcePerson()`).

POINTER / RELATIONSHIP UPDATES

Marking someone deceased does not sever any pointers by itself; divorcing does, via `disconnectSpouses()`.

VALIDATION

Divorce requires an existing spouse (`hasSpouse()`); marking deceased twice is rejected with `PERSON_DECEASED`.

SPECIAL CASES

The submenu loops after each action so multiple attributes can be changed in one visit, exiting only on "Back".

COMPLEXITY

$O(1)$ for every operation.

DESIGN JUSTIFICATION

Grouping all mutation operations behind one "Update Person" gate (rather than scattering them across the main Person Menu) keeps read-only navigation options visually separated from state-changing ones.

6.17 Export Family Tree to XML

PURPOSE

Produce a durable, human-readable snapshot of a chosen family tree.

DATA STRUCTURES USED

`Person` (read-only, via recursion), `ofstream`.

PROCEDURE

`createXML()` lets the user pick a tree, opens a file stream, and calls `writeXML(root, file, 0)`.

INTERNAL WORKING

`writeXML()` recursively emits one `<Person>` element per node — attributes for id/name/gender/status, then nested elements for spouse and each child — indenting each level via `writeIndentation()`.

VALIDATION

Special XML characters in names are escaped via `escapeXML()` before being written, so a name containing `&`, `<`, or quotes cannot corrupt the document.

SPECIAL CASES

A person's spouse is written as a reference element rather than being recursively expanded again, avoiding infinite mutual recursion between spouses.

COMPLEXITY

$O(n)$, n = people in the exported tree.

DESIGN JUSTIFICATION

XML was chosen over a flat text dump because its nesting maps directly onto the parent/child recursion already used elsewhere in the program, requiring no extra data transformation step.

6.18 Print Family Tree

PURPOSE

Display a chosen tree as an indented, human-readable outline directly in the console.

PROCEDURE

`viewFamilyTree()` lets the user pick a tree by ID, then calls `printFamilyTree(root)`, which delegates to `printPersonRecursive()`.

INTERNAL WORKING

Uses a running prefix string and an "is this the last child at this level" flag to draw connector lines, the same visual technique used by common directory-tree printers.

SPECIAL CASES

An empty forest, or a tree ID that does not exist, is reported without attempting to print.

COMPLEXITY

$O(n)$, n = people in the tree.

DESIGN JUSTIFICATION

A quick on-screen view complements the XML export: XML is for archiving/sharing, the console print is for a fast sanity check while working.

6.19 Exit Program

PURPOSE

Cleanly terminate the session.

PROCEDURE

Choosing `0` from the main menu breaks the loop; `clearHistory()` frees every remaining `HistoryNode`, then `freeAllTrees()` recursively deletes every `Person` in every tree before the process returns.

SPECIAL CASES

Exit is graceful even if trees or history are already empty — every cleanup routine is null-safe.

COMPLEXITY

$O(n + d)$, n = total people, d = history depth.

DESIGN JUSTIFICATION

Explicit cleanup on exit, rather than relying on the OS to reclaim memory, keeps the program well-behaved for tools like Valgrind and is good discipline for a manual-memory-management (non-smart-pointer) codebase.

Relationship Constraints

7.1 Parent Constraints

- **One Father Rule** — enforced structurally (single pointer) and by `hasFather()` / `ALREADY_EXISTS` before assignment.
- **One Mother Rule** — symmetric to the above via `hasMother()`.
- **Parent Must Exist** — both the child and the candidate parent are null-checked before any pointer is touched.
- **Duplicate Parent Prevention** — re-registering an already-set father/mother is rejected rather than silently overwritten.
- **Parent Pointer Updates** — `child->fromFamily.father/mother` set exactly once per slot.
- **Child Linked List Updates** — the child is simultaneously appended into the parent's `firstChild/lastChild` chain, keeping both directions of the relationship consistent in one operation.

7.2 Child Constraints

- **Duplicate Child Prevention** — `childAlreadyExists()` walks the sibling chain before insertion.
- **Parent Consistency** — when a spouse exists, a new child is registered under both parents in the same operation, never under just one with the other left dangling.
- **Linked List Insertion** — always an O(1) append at `lastChild`.
- **Relationship Synchronization** — the child's own `fromFamily` pointers and the parents' `extendFamily` chains are updated together, so no relationship is ever one-sided.

7.3 Marriage Constraints

- **One Active Spouse** — a second marriage is blocked with `ALREADY_EXISTS` unless the existing one is first resolved.
- **Bidirectional Spouse Pointers** — set and cleared together, never independently.
- **Existing Marriage Handling** — `resolveExistingMarriage()` presents an explicit choice instead of failing silently.
- **Divorce** — clears both spouse pointers via `disconnectSpouses()`, leaving all other relationships (parents, children) untouched.
- **Deceased Spouse** — sets `status = DECEASED` on the former spouse in addition to disconnecting the pointer.
- **Cancel Operation** — aborts the entire spouse-registration flow with no pointer changes at all.
- **Duplicate Prevention** — the same two people cannot be connected twice; the second attempt is caught by the "already has a spouse" check.
- **Self Marriage Prevention** — `isSamePerson()` rejects a person being registered as their own spouse.

7.4 Search Constraints

- **Duplicate Names** — every match is returned, not just the first; the user chooses among them.
- **Multiple Results** — presented as a numbered list via `chooseSearchResult()`, with an explicit cancel option.
- **Temporary Search List** — `SearchNode` results (and the internal visited-tracking list) are fully destroyed after each search, independent of whether the user selected a result.

7.5 Navigation Constraints

- **History Stack** — only pushed when a person's menu is opened *from* a known predecessor (a newly created relative), not on every relationship view — actually, relationship views (Parents/Children/Siblings/Spouse) push as well, since each hop away from a person is a legitimate "came from" point.
- **Back Navigation** — "Back" (option 8) simply unwinds the current call frame; it does not touch the history stack, which is reserved exclusively for "Previous Person".
- **Root Handling** — entry points with no logical predecessor (new tree registration, a person reached via search) clear the history stack instead of pushing to it, preventing an unrelated earlier session's entries from being popped by mistake.

7.6 Update Constraints

- **Name Update** — free-form re-validation identical to initial name entry; no uniqueness constraint on names (IDs remain the unique key).
- **Life Status Update** — marking an already-deceased person deceased again is rejected (`PERSON_DECEASED`) rather than treated as a no-op success.
- **Divorce Rules** — requires an existing spouse; divorcing a person with no spouse returns `PERSON_NOT_FOUND` rather than silently succeeding.

Input Validation

Concern	Mechanism
Integer Validation	<code>inputInteger()</code> uses <code>cin.fail()</code> detection, clears the stream, and discards the bad line before reprompting — a non-numeric entry can never crash the read loop.
Name Validation	<code>inputName()</code> rejects empty input and any character outside letters, spaces, apostrophes, and hyphens.
Menu Validation	<code>inputChoice(min, max, prompt)</code> wraps <code>inputInteger()</code> and loops until the value falls within the given inclusive bounds.
Range Checking	Every menu (main menu, person menu, sub-menus) is bounded explicitly by its own <code>min / max</code> call, so no menu can accept an option it does not actually offer.
Duplicate ID Prevention	IDs are never user-supplied; they come from a single incrementing counter (<code>nextPersonID</code>), making collisions structurally impossible.
Invalid Relationship Prevention	Every relationship-creating action passes through a dedicated <code>validate...Registration()</code> function first (Chapter 7) before any pointer is touched.
Null Pointer Validation	Essentially every function that accepts a <code>Person*</code> checks for <code>nullptr</code> as its first statement and returns a descriptive <code>Result</code> rather than dereferencing blindly.
Invalid Navigation Handling	<code>popHistory()</code> returns <code>nullptr</code> on an empty stack, and the caller checks for that before reassigning <code>currentPerson</code> , so "Previous Person" with nothing to go back to reports that clearly instead of corrupting the session.

Design Decisions

9.1 Why Family Forest?

A single mandatory tree would force every person into one ancestry from the moment they are created. A forest lets independent families exist side by side and only become connected when the user actually marries or links people across them — matching how real family relationships actually form.

9.2 Why Linked Lists?

The number of children, trees, search results, and history entries is unknown and unbounded at compile time. Linked structures grow and shrink one node at a time with $O(1)$ insertion/removal at the relevant end, without the reallocation and copying overhead of a resizable array.

9.3 Why Parent Pointers?

A direct pointer gives $O(1)$ access to a known relationship instead of a search. Using a single pointer (not a list) for father and mother also makes the "at most one of each" rule a property of the type itself, not just of the validation code around it.

9.4 Why Spouse Pointers?

Identical reasoning to parent pointers: $O(1)$ access, and a single pointer makes monogamy structurally enforced rather than merely policy-enforced.

9.5 Why Siblings are NOT Stored?

See 6.14 — storing siblings separately would create a second source of truth that could desynchronize from the parent/child pointers. Deriving them on demand guarantees correctness by construction.

9.6 Why an Intrusive Child List instead of a Wrapper/Array?

An array would need periodic resizing and would not naturally support $O(1)$ removal from the middle. A generic wrapper node would double allocations. Embedding `leftSibling / rightSibling` directly in `Person` avoids both costs.

9.7 Why First Child and Last Child Pointers?

Keeping only `firstChild` would force an $O(n)$ walk to find the append point for every new child. Adding `lastChild` turns that into $O(1)$ at the cost of two extra pointer fields on `ExtendFamily`.

9.8 Why SearchNode?

Search results need their own temporary, disposable collection that is entirely separate from the permanent family graph, so that browsing or discarding results can never accidentally mutate a real relationship pointer.

9.9 Why History Stack?

"Previous Person" is inherently a last-in-first-out operation — you want to undo the most recent navigation step first. A stack matches that access pattern exactly, with $O(1)$ push and pop.

9.10 Why Recursion?

The underlying data (a tree/graph of people) is itself recursively defined — a person's children are, from a traversal standpoint, small trees in their own right. Recursive functions for printing, XML export, and search mirror that structure directly, which keeps each function short and its correctness easy to reason about by induction on tree depth.

9.11 Why XML Export Uses Recursion?

XML's element nesting is structurally identical to the parent/child nesting already being traversed for printing and search, so the same recursive walk, with a different "visit" action (write a tag instead of print a line), produces the export with no separate flattening step.

9.12 Why Dynamic Memory Allocation?

The number of people, trees, search results, and history entries is unknown until runtime and changes constantly as the user interacts with the program. `new`-allocated nodes let every structure grow exactly as needed and be explicitly freed when no longer required.

9.13 Why Global Variables?

Only truly program-wide state — the forest head, the navigation-history top, and the two ID counters — is global. This is intentionally minimal: everything else (including "current person") is passed explicitly through parameters, keeping most of the call graph free of hidden shared state.

9.14 Why Unique Person IDs?

Names are not unique (the system explicitly supports duplicate names, see 7.4), so a separate, guaranteed-unique numeric ID is required for unambiguous lookup, especially for "Search by ID" and for distinguishing two same-named search results from each other.

9.15 Why Menu-Driven Design?

A bounded, numbered menu at every step keeps input validation simple (a single integer range check) and makes the tool usable without needing to remember command syntax — appropriate for a console application aimed at demonstrating data-structure behaviour rather than a scripting/automation tool.

9.16 Why Temporary Search Results?

Search is, by nature, a transient query — its output should not linger in memory or interfere with later searches. Building a fresh `SearchNode` list per search and destroying it immediately after use keeps each search operation self-contained.

9.17 Why Bidirectional Relationships?

Every meaningful relationship in the system is set on both sides at once (child $\leftrightarrow$ parent, spouse $\leftrightarrow$ spouse) so that traversal works correctly starting from *either* person — a prerequisite for the full graph search described in 6.8/6.9, and for features like View Parents and View Children to be mirror images of each other.

Complexity Analysis

Operation	Best Case	Worst Case	Reasoning
Register root person / create tree	O(1)	O(1)	Direct allocation and head insertion into the forest list.
Create new father / mother / spouse / child	O(1)	O(k)	O(1) allocation and pointer assignment; O(k) only for the child-duplicate check against k existing siblings.
Link existing father / mother / spouse	O(1)	O(n)	Best case the target is the very first node visited; worst case the full connected graph of n people must be walked.
Search by ID	O(1)	O(n)	Full father/mother/spouse/children graph walk with a visited guard, short-circuiting on first match.
Search by Name	O(1)	O(n + r ²)	Same graph walk (O(n)); duplicate-guarded insertion of r results adds up to O(r ²) in the worst case.
View Parents	O(1)	O(1)	At most two direct pointer reads.
View Children	O(1)	O(k)	Linear walk of the sibling chain, k = number of children.
View Siblings	O(1)	O(k)	Linear walk of the shared parent's child chain.
View Spouse	O(1)	O(1)	Single direct pointer read.
Previous Person (push / pop)	O(1)	O(1)	Stack head insertion / removal.
Update Person (name / status / divorce)	O(1)	O(1)	Direct field mutation and, for divorce, a single pair of pointer clears.
Print Family Tree	O(n)	O(n)	Every node in the tree is visited exactly once.
Export to XML	O(n)	O(n)	Same single full traversal, writing instead of printing.
Delete Family Tree	O(n)	O(n)	Every node in the tree must be individually freed.
Program exit cleanup	O(n + d)	O(n + d)	Free every person across every tree (n) plus every remaining history entry (d).

n = total connected people in the graph at the time of the operation. k = number of children of the relevant parent. r = number of matches returned by a name search. d = current depth of the navigation-history stack.

Libraries Used

Library	Purpose	Functions Used	Why Required	Effect if Removed
<iostream>	All console input and output.	cin , cout , stream extraction/insertion operators.	The entire application is a text-menu console program; every prompt, every result, and every piece of user input passes through this library.	No menu could be displayed and no input could be read — the program could not run at all.
<string>	Storage and manipulation of names, gender labels, and status text.	std::string , comparison and concatenation operators.	Names and genders are variable-length text with no natural fixed size; std::string handles growth, comparison, and safe copying automatically.	Would require reverting to fixed-size C-style character arrays, reintroducing manual buffer-overflow and null-termination risk throughout the codebase.
<fstream>	Writing the XML export file.	std::ofstream , stream insertion into a file.	XML export is the program's only durable output; file streams are the standard C++ mechanism for writing to disk.	The Export to XML feature would be entirely unimplementable.
<limits>	Recovering cleanly from invalid integer input.	std::numeric_limits<streamsize>::max() , used with cin.ignore() .	After a failed numeric read, the invalid characters remain in the input buffer; this call discards them so the next prompt is not immediately fed the same bad input.	A non-numeric entry at any menu would put the input stream into a stuck failure state, causing an infinite prompt loop.
<cctype>	Character classification and case conversion.	isalpha() , tolower() , toupper() .	Used to validate that a name contains only letters (plus spaces/apostrophes/hyphens), to normalize gender input, and to implement fully case-insensitive name search.	Name validation and the case-insensitive search feature (Chapter 6.9) would have to be hand-rolled with raw character-code arithmetic, which is both less readable and more error-prone.

Conclusion

The Family Tree Management System demonstrates that a rich, interconnected real-world structure — a family, with its parents, spouses, children, and siblings — can be modelled entirely with a small set of classic linked data structures and recursive algorithms, without any external database. Every relationship in the system reduces to a plain pointer, every "list" of relatives is an intrusive linked chain rather than a separate container, and every derived view (siblings, printed trees, XML exports, search results) is computed on demand from that single source of truth rather than duplicated and risked going stale.

Objectives achieved: multi-tree ("forest") support with cross-tree linking; full create-or-link relationship registration for fathers, mothers, spouses, and children; correct handling of divorce and deceased spouses; consistent, undoable navigation via a history stack; and a name/ID search that is both case-insensitive and capable of reaching every connected person in the graph, not merely those who happen to be a tree's root.

Major functionalities: family-forest registration, relationship management with full validation, person navigation with history, dual-mode search, in-place person updates, and XML export.

Data structures implemented: `Person` (with embedded `FromFamily / ExtendFamily` and an intrusive sibling list), `FamilyTree` forest list, `HistoryNode` navigation stack, and `SearchNode` transient result lists.

Future improvement possibilities: persistent storage between sessions (e.g. re-importing a previously exported XML file); support for more complex family structures such as multiple marriages or adoption; a graphical tree renderer in place of the indented text view; and a formal automated test suite covering the relationship-validation rules in Chapter 7.